



Cloud Remote Access

Last updated: Sunday, September 07, 2025

This content applies to Cumulocity 2025 and to all subsequent releases.

Specifications contained herein are subject to change and these changes will be reported in subsequent versions.

Copyright © 2025 Cumulocity GmbH.

The name Cumulocity GmbH and all Cumulocity GmbH product names are either trademarks or registered trademarks of Cumulocity GmbH and/or its subsidiaries and/or its affiliates and/or their licensors. Other company and product names mentioned herein may be trademarks of their respective owners.

This software may include portions of third-party products. Third-party terms are set out in a 3rd-party-licenses file linked to or included with each installation package.

Table of Contents

Table of Contents	3
INTRODUCTION	4
HOW CLOUD REMOTE ACCESS WORKS	4
BROWSER-BASED SUPPORT FOR VNC, SSH AND TELNET	4
PASSTHROUGH: SUPPORT FOR NATIVE CLIENTS AND PROPRIETARY PROTOCOLS	5
SETTING UP CLOUD REMOTE ACCESS	6
PREREQUISITES	6
SET UP YOUR DEVICE	6
USER AUTHORIZATION	6
ACCESSING CLOUD REMOTE ACCESS	6
MANAGING ENDPOINTS	7
To configure a new remote device	7
To add a remote access endpoint via VNC	7
To add a remote access endpoint via SSH	8
To add a remote access endpoint via Telnet	11
To add a remote access endpoint via Passthrough	12
To edit an endpoint	12
To delete an endpoint	12
To connect to an endpoint	12
Auto-saving host key functionality	13
AUDIT LOGS	13
TROUBLESHOOTING	14
CLOUD REMOTE ACCESS API	15
OVERVIEW	15
CLIENT IMPLEMENTATION	15
Example	16
WEBSOCKET HTTP HEADERS	17
SENDING AND RECEIVING TRAFFIC	17
DEVICE AGENT IMPLEMENTATION	17
SUPPORTED OPERATION	17
CONNECT OPERATION	17
CONNECTING TO A NEW ENDPOINT	18
OPERATING A CONNECTED ENDPOINT	18
DISCONNECTING AN ENDPOINT	18
RECOMMENDATIONS	18

INTRODUCTION

The Cumulocity Cloud Remote Access feature offers a seamless and secure way to connect to your devices using either a browser-based remote desktop or an SSH connection. Devices can be directly accessed through the Cumulocity platform's web-based user interface without exposing any ports to a public network, thereby eliminating the need for additional software or complex VPN infrastructure. Alternatively, you can also establish a secure TCP tunnel directly to a local machine.

Leveraging these remote access and management capabilities allows for effective configuration and troubleshooting of devices, such as industrial machines, IoT gateways, or network infrastructure. This includes, for example, the following scenarios:

1. Remote assistance and troubleshooting of machines by accessing the Human Machine Interface (HMI) via VNC.
2. Access local UIs like the configuration UI of a NodeRed server to manage your flows or access the stream of your CCTV camera.
3. Directly access an SSH server in a private network from the browser via WebSSH without requiring any additional software. The connection is easy to set up which makes it very convenient particularly when working with multiple devices.
4. Natively connect to an SSH server in a private network with any local client like PuTTY or OpenSSH. This provides more advanced functionalities compared to WebSSH, like file copy via SCP, while providing better performance and latency.
5. Remotely develop and debug logic and scripts deployed on top of your devices by using the Codesys IDE or Visual Studio Code using CRA with the Cumulocity CLI acting as a [local proxy](#).

HOW CLOUD REMOTE ACCESS WORKS

This versatile feature offers two primary connection methods:

1. **Direct device access:** Establish a seamless connection to devices directly linked to Cumulocity.
2. **Gateway-enabled remote access:** Leverage a connected device as a gateway to access any device that is reachable within its local area network, expanding your reach to manage multiple devices through a single entry point.

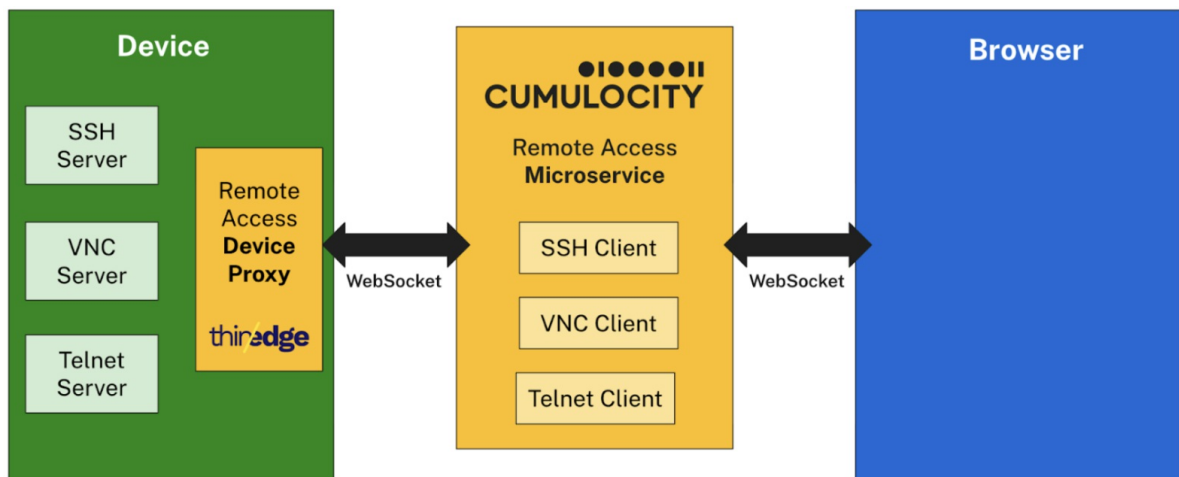
The connection is always initiated by the device. The feature operates through a microservice running within Cumulocity, which tunnels all protocols through a secure WebSocket connection and manages authentication without the need to open any port. This approach provides a level of security comparable to traditional VPN tunnels while offering greater simplicity and ease of use.

Key security features include:

1. **TLS encryption** for all connections to remote devices.
2. **RBAC** to prevent unauthorized personnel from accessing devices and making changes to critical parameters.
3. Auditability provided through **audit logs** which get automatically created for each remote session.

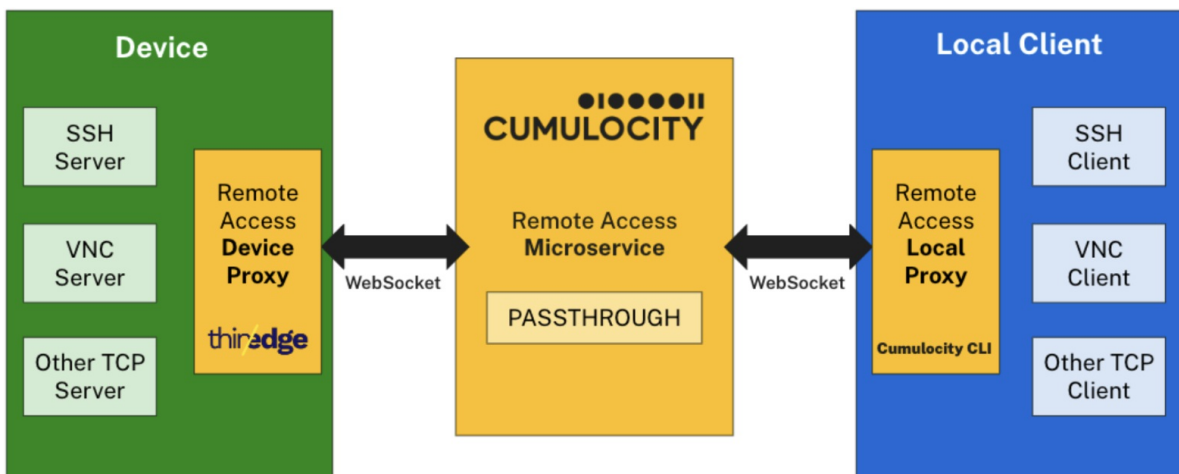
To leverage Cloud Remote Access, your device needs to be enabled by installing [thin-edge.io](#). Thin-edge.io is designed to fully integrate with this feature and all other Device Management functionalities provided by Cumulocity. By combining Cumulocity's Cloud Remote Access with thin-edge.io, you can achieve a secure, efficient, and user-friendly remote device management solution that scales with your IoT deployment.

BROWSER-BASED SUPPORT FOR VNC, SSH AND TELNET



A common use case is the need to remotely access a device configuration interface using SSH, VNC, or Telnet. For these protocols, the CRA feature provides a convenient browser-based client that is integrated in the user interface of Cumulocity.

PASSTHROUGH: SUPPORT FOR NATIVE CLIENTS AND PROPRIETARY PROTOCOLS



While it is suitable for many scenarios to access the server through a web terminal with connections terminating at the Cloud Remote Access microservice, it may not meet the requirements of more complex use cases. For these scenarios, Cumulocity offers a passthrough option that enables the use of native clients by tunnelling packets to a proxy running locally on your machine. This allows you to natively connect to the SSH server from your local machine, access the local Web UI, or tunnel an HTTP server that is running on your device. Basically any TCP port can be bridged that way, not only SSH, VNC, or HTTP traffic.

The easiest way to setup a local proxy is via the Cumulocity CLI, which includes a [built-in local proxy](#) supporting the following transport mediums:

- Unix socket
- TCP port
- Standard input/output (stdio)

SETTING UP CLOUD REMOTE ACCESS

Cloud Remote Access is available in the [Device Management application](#).

PREREQUISITES

✔ REQUIREMENTS

To use Cloud Remote Access, you need

- a Cloud Remote Access compatible gateway connected to your Cumulocity account.
- “Remote access” permission granted to the tenant user.
- the Cloud Remote Access microservice included into your subscription plan.

SET UP YOUR DEVICE

To configure your device for compatibility with the Cloud Remote Access functionality, you must install thin-edge.io. Thin-edge.io is fully integrated with Cloud Remote Access and can be easily deployed on any Linux-based device, eliminating the need for any custom integration.

Furthermore, devices can report their supported protocols using [SmartREST template 150](#) which offers two significant advantages:

1. It prevents the display of unsupported protocols in the selection modal, streamlining the user experience and reducing potential confusion.
2. It enables administrators to restrict the use of potentially vulnerable protocols, such as Telnet, thereby enhancing the overall security of your deployment.

USER AUTHORIZATION

The Cloud Remote Access feature is a powerful tool that requires careful management. Due to its potential for significant system impact, access should be restricted to experienced administrators who fully understand its capabilities and risks.

To grant access to qualified users:

1. Navigate to **Accounts > Roles** in the Administration application.
2. Click **Add global role** at the top right.
3. Create a new role with the following details:
 - **Name:** “Cloud Remote Access Role”
 - **Permissions:** Enable “Admin” permission for “Remote Access”

ACCESSING CLOUD REMOTE ACCESS

In the Device Management application in the Cumulocity platform, click **All devices** in the **Devices** menu and select the desired gateway from the device list.

When you open the gateway you will find the **Remote access** tab in its tab list.

Linux MAC 8AF8AF9F061A

Add endpoint More...

Endpoint	Host : Port	Protocol		
Passthrough connection	127.0.0.1 : 22	PASSTHROUGH		
SSH Connection public/private	127.0.0.1 : 22	SSH		
Telnet connection	127.0.0.1 : 23	TELNET		
VNC Connection	127.0.0.1 : 5900	VNC		

In the **Remote Access** tab, you can configure devices for remote control, so-called “endpoints”, and connect to remote devices.

Connections can be established to the gateway itself (localhost) or to any device in the local area network reachable by the device.

MANAGING ENDPOINTS

The “endpoint” is the IP address and port of the VNC, SSH or Telnet server running on the remote device. The IP address and port must be reachable from the gateway.

To configure a new remote device

1. Click **Add endpoint** at the right of the top menu bar.
2. Enter a name for the new endpoint and select the protocol to be used.
3. Follow the descriptions below for the protocol-specific settings.

INFO

To be able to configure an endpoint, you need ADMIN permission for “Remote access” and “Device control”.

To read data, a READ permission is sufficient. For more information on permissions, refer to [Managing permissions](#).

To add a remote access endpoint via VNC

1. Enter the host (IP address or hostname) and the port of the server.
2. Select a sign-in method. If you select “Password only”, provide the password for the VNC server.
3. Click **Save** to add the endpoint.



REMOTE ACCESS ENDPOINT

Name

e.g. My remote access endpoint (required)

Protocol

VNC

Host

127.0.0.1

Port

5900

Sign-in method

No password

Cancel

Save

Once the connection is established, a new browser tab will open displaying the front screen or operating panel of the remote device you are connected to. The top bar of the screen will show "starting VNC handshake" when the process is starting.

INFO

The following versions of the VNC protocol are currently supported:

- RFB 003.003
- RFB 003.007
- RFB 003.008

The functionality has been tested on the following VNC servers:

- Real VNC 5.3.2
- Tiger VNC 1.6.0/1.7.0
- TightVNC 1.3.9
- EfonVNC 4.2
- Vino

[To add a remote access endpoint via SSH](#)

1. Enter the host (IP address or hostname) and the port of the server.
2. Select a sign-in method.

Username and password: If this method is selected, it is mandatory to enter a username and password.



REMOTE ACCESS ENDPOINT

Name

e.g. My remote access endpoint (required)

Protocol

SSH

Host

127.0.0.1

Port

22

Sign-in method

Username and password

Username

e.g. my_username (required)

Password

e.g. my_password

Host key 

e.g. ssh-rsa AAAAB3NzaC...1kc3MAAACB== user@exan

 **Drop file here**

Cancel

Save

Public/private keys: Automatically generate public and private keys or simply paste pre-generated keys. The keys can also be uploaded from a file.



REMOTE ACCESS ENDPOINT

22

Sign-in method

Public/private keys

Username

e.g. my_username (required)

 **Generate public and private keys**

Private key [?](#)

e.g. -----BEGIN RSA PRIVATE KEY-----MIIEpAIBAAK...VC

 **Drop file here**

Public key [?](#)

e.g. ssh-rsa AAAAB3NzaC...1kc3MAAACB== user@exan

 **Drop file here**

Host key [?](#)

e.g. ssh-rsa AAAAB3NzaC...1kc3MAAACB== user@exan

 **Drop file here**

Cancel

Save

INFO

The public key must be installed on the remote device as authorized key.

- Optionally, you can also add a host key to ensure connection to the correct device. This key can also be uploaded from a file.
- Click **Save** to add the endpoint.

The following formats are supported when adding new keys:

- OpenSSHv1
- OpenSSHv2
- PEM
- SSH2

The following algorithms are supported when adding new keys:

- RSA
- DSA
- ECDSA
- ED25519

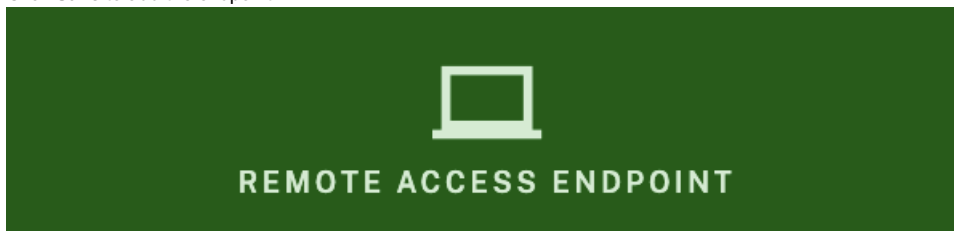
i INFO

Limitations:

- Character support: International characters are not supported. Only a limited set of control characters is functional.
- Input restrictions: Mouse movements are not supported.
- Protocol compatibility: SSH version 1 is not supported; only SSH version 2 is available.
- Display behavior: Text reflow does not occur when the window width changes.

To add a remote access endpoint via Telnet

1. Enter the host (IP address or hostname) and the port of the server.
2. Click **Save** to add the endpoint.



Name

e.g. My remote access endpoint (required)

Protocol

TELNET

Host

127.0.0.1

Port

23

Cancel

Save

i IMPORTANT

Telnet is considered to be an insecure protocol lacking built-in security measures. For network communication in a production environment we highly recommend you to use the SSH protocol instead.

To add a remote access endpoint via Passthrough

1. Enter the host (IP address or hostname) and the port of the server.
2. Click **Save** to add the endpoint.



REMOTE ACCESS ENDPOINT

Name

Protocol

Host

Port

Cancel

Save

Visit the [Cumulocity CLI](#) documentation to learn more about how to set up the passthrough connection from the device to your local machine using the local proxy that is embedded in the CLI.

To edit an endpoint

To edit an endpoint, click the menu icon  at the right of the respective entry and select **Edit** from the context menu.

To delete an endpoint

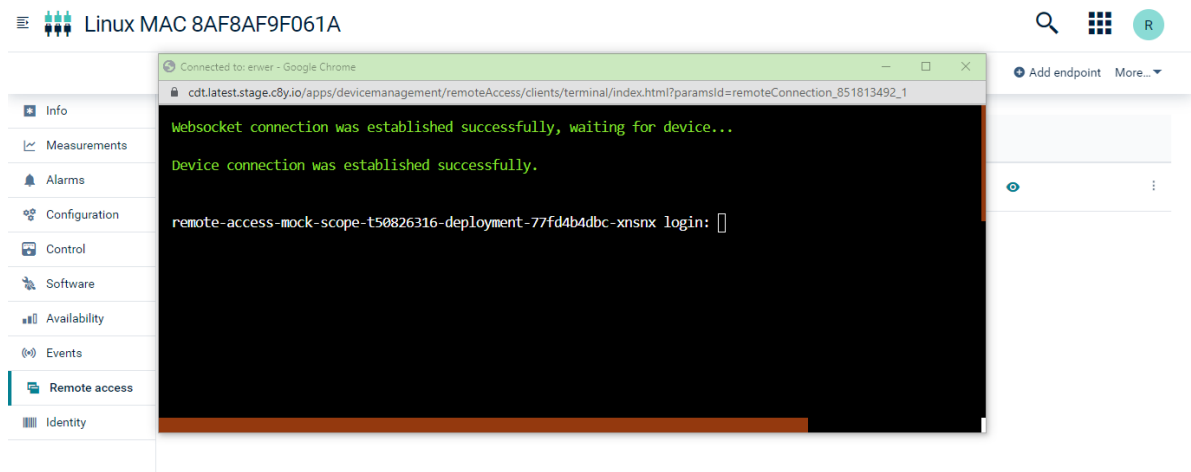
To delete an endpoint, click the menu icon  at the right of the respective entry and select **Remove** from the context menu.

INFO

An active connection will not be terminated automatically after the endpoint was deleted.

To connect to an endpoint

To connect to configured endpoints, select an endpoint in the **Remote access** tab and click **Connect**. The connection to the configured remote device is established and the screen is shared in the client area.



To terminate the connection, click **Disconnect**.

Auto-saving host key functionality

A host key is a public key of the server which is generated when an SSH server is installed. It is used to verify the identity of the server.

By enabling the auto-saving host key functionality you will no longer need to enter the host key after each connection. Instead, the host key can be automatically saved after the first successfully established connection to a remote access endpoint.

In order to enable the auto-save host key functionality, navigate to the **Remote access** page under the **Settings** menu in the **Administration** application. Activate the checkbox and then click **Save**.

AUDIT LOGS

For each gateway device, audit logs are available.

The audit logs can be found in the **Control** tab of the gateway device.

Linux MAC 8AF8AF9F061A		Search	Grid	Refresh
All statuses		Realtime	Reload	More...
	Operation	Date created		
Info	Opening remote access tunnel to 'erwer'	6 Feb 2023, 11:29:26	✓	⋮
Measurements	Opening remote access tunnel to 'erwer'	6 Feb 2023, 11:28:24	✓	⋮
Alarms	Opening remote access tunnel to 'erwer'	3 Feb 2023, 12:20:42	✓	⋮
Configuration	Opening remote access tunnel to 'erwer'	3 Feb 2023, 12:19:08	✓	⋮
Control	Opening remote access tunnel to 'erwer'	3 Feb 2023, 12:15:18	✓	⋮
Software	Opening remote access tunnel to 'erwer'	3 Feb 2023, 12:12:54	✓	⋮
Availability	Opening remote access tunnel to 'erwer'	3 Feb 2023, 12:00:21	✓	⋮
Events	Opening remote access tunnel to 'erwer'	3 Feb 2023, 11:59:42	✓	⋮
Remote access				
Identity				

For each connection, the Cloud Remote Access microservice creates an operation in scope of the current user. Then the operation will be updated by the device to reflect the current status. Finally, the operation will have a status of SUCCESSFUL or FAILED.

TROUBLESHOOTING

Endpoints cannot be set up

If you cannot set up new endpoints, check if you have sufficient permissions.

To set up new endpoints, you need ADMIN permission for “Device control” to be able to register a device and ADMIN permission for “Remote access” to be able to add an endpoint.

For more information on permissions, refer to [Managing permissions](#).

Connection fails

The connection via a gateway to a remote VNC, SSH or Telnet server can fail because of network problems. In this case you must contact your network administrator.

Unsupported protocol version

In case of Real VNC, if you get an error message stating that you are using an unsupported protocol version (for example 005.00x), try the following workaround:

1. Open VNC.
2. Navigate to **Options**.
3. Select the **Export** tab.
4. Search for **Protocol version**.
5. Enter “3.8” as protocol version.

CLOUD REMOTE ACCESS API

Cumulocity provides a public API for the Cloud Remote Access feature. This API is designed for two different integration scenarios:

1. Development of **device-side components** which enable access to services like VNC, SSH, Telnet, or arbitrary TCP-based protocols (passthrough) using the device as a gateway.
2. Integration and development of **native TCP protocol clients or forwarding proxies**, for example as an alternative to the commonly used [C8Y Cli passthrough support](#).

OVERVIEW

```
sequenceDiagram
    actor U as User participant CL as Client participant CRA as CRA microservice participant CORE as Cumulocity Core
    participant DA as Device agent participant S as External endpoint
    (SSH, Telnet, VNC...) U -->> CL: Start Client session
    (Example: SSH) CL -->> CRA: Connect WebSocket
    to CRA configuration activate CRA
    CRA -->> CORE: Create c8y_RemoteAccessConnect
    operation CORE -->> DA: Push operation
    DA -->> CORE: Mark operation as EXECUTING
    DA -->> CRA: Connect WebSocket to connection key
    of operation activate DA loop
    DA <-->> S: Forward data packets to WebSocket
    and vice versa end
    DA -->> CORE: Mark operation as SUCCESSFUL deactivate CRA deactivate DA
```

The diagram above illustrates the end-to-end integration between a user's client and the corresponding external server via Cloud Remote Access (CRA). In such a setup, we distinguish between the following participants and components:

- A **Cumulocity user**, who wants to access an external service like a web server or SSH that is only reachable from a remote device.
- The **client**. This often is a shell in the Cumulocity UI; alternatively it can be a forwarding proxy like [C8Y Cli](#) or even a custom client application seeking access to the external service.
- The **Cloud Remote Access (CRA)** microservice at `/service/remoteaccess`.
- The **Cumulocity Core platform** (see also the [Cumulocity OpenAPI Specification](#)).
- The **device agent**. A common open-source agent is [thin-edge.io](#).
- An arbitrary **external endpoint**, typically a SSH, HTTP, Telnet or VNC server.

In the following, we now describe both how to implement a device-side agent and a client application that connects to the device-side agent via CRA.

CLIENT IMPLEMENTATION

Each device supporting Cloud Remote Access uses a fragment called `c8y_RemoteAccessList` to hold a list with the configured endpoints that can be accessed via this device.

An entry in the `c8y_RemoteAccessList` fragment holds the following fragments:

Field	Data Type	Mandatory	Details
<code>id</code>	String	Yes	Identifies the configuration of the remote access entry.
<code>port</code>	Number (Integer)	Yes	The port number of the service on the remote device.

Field	Data Type	Mandatory	Details
<code>name</code>	String	Yes	A user-friendly name for the remote access entry.
<code>hostname</code>	String	Yes	The hostname or IP address of the service to be accessed remotely.
<code>protocol</code>	String	Yes	The protocol used to access the service. Either <code>PASSTHROUGH</code> , <code>TCP</code> , <code>VNC</code> , <code>TELNET</code> or <code>SSH</code> .
<code>credentials</code>	Object	Yes	An object containing authentication credentials for accessing the remote service.
<code>credentials.privateKey</code>	String	See details	Mandatory if <code>credentials.type</code> is <code>KEY_PAIR</code> or <code>CERTIFICATE</code> . Otherwise, not applicable.
<code>credentials.password</code>	String	See details	Mandatory if <code>credentials.type</code> is <code>PASS_ONLY</code> or <code>USER_PASS</code> . Otherwise, not applicable.
<code>credentials.certificate</code>	String	See details	Mandatory if <code>credentials.type</code> is <code>CERTIFICATE</code> . Otherwise, not applicable.
<code>credentials.publicKey</code>	String	No	Public key for authentication, if applicable.
<code>credentials.hostKey</code>	String	No	Host key for authentication, if applicable.
<code>credentials.type</code>	String	Yes	Type of credentials. Possible values are: - <code>NONE</code> : No credentials required. - <code>PASS_ONLY</code> : Only <code>password</code> is required. - <code>USER_PASS</code> : Both <code>username</code> and <code>password</code> are required. - <code>KEY_PAIR</code> : Both <code>username</code> and <code>privateKey</code> are required. - <code>CERTIFICATE</code> : <code>username</code> , <code>privateKey</code> , and <code>certificate</code> are required.
<code>credentials.username</code>	String	See details	Mandatory if <code>credentials.type</code> is <code>USER_PASS</code> , <code>KEY_PAIR</code> , or <code>CERTIFICATE</code> . Otherwise, not applicable.

Example

```

"c8y_RemoteAccessList": [
  {
    "hostname": "localhost",
    "protocol": "PASSTHROUGH",
    "credentials": {
      "privateKey": null,
      "password": null,
      "certificate": null,
      "publicKey": null,
      "hostKey": null,
      "type": "NONE",
      "username": null
    },
    "port": 33123,
    "name": "My HTTP Echo Server",
    "id": "1"
  }
]

```


In the example above a local HTTP echo server is reachable from the device at `http://localhost:33123`. The configuration ID is `1`. To connect a client application, the client must open a WebSocket to the following URL.

```
wss://<tenant domain>/service/remotearraccess/client/<device id>/configurations/<configuration id>
```

WEBSOCKET HTTP HEADERS

The Cloud Remote Access service uses the binary WebSocket subprotocol, regardless of which protocol is requested by the client. Hence, we recommend pinning the WebSocket subprotocol to `binary` using the `Sec-WebSocket-Protocol` header. Additionally, a valid authorization header is required, see [Authentication](#) in the Cumulocity OpenAPI Specification.

```
Sec-WebSocket-Protocol: binary
Authorization: <auth header>
```

SENDING AND RECEIVING TRAFFIC

Once the WebSocket connection is established, traffic for the endpoint can simply be sent to the WebSocket. Data from the endpoint can be consumed by reading from the WebSocket. To implement a local forwarding proxy, data from a local server socket must be written to the WebSocket and vice versa.

DEVICE AGENT IMPLEMENTATION

This section describes how to implement a device agent deployed on a gateway. The device agent is responsible for creating the device part of the tunnel between a TCP/IP connection at private network and the secure device WebSocket endpoint.

SUPPORTED OPERATION

A device agent that implements an integration with Cloud Remote Access must consume `c8y_RemoteAccessConnect` operations. They are used to inform the device about tunnel connections it should establish. In order to mark your device being capable of handling them, it needs to include `c8y_RemoteAccessConnect` as a supported operation in its managed object:

```
"c8y_SupportedOperations" : [
...
"c8y_RemoteAccessConnect",
...
]
```

CONNECT OPERATION

This operation is created when the application generates a connect request. The operation is then sent to the device agent, which establishes a connection between the WebSocket endpoint at the server and the local network endpoint.

Example of an `c8y_RemoteAccessConnect` operation:

```
{
...
"c8y_RemoteAccessConnect": {
  "hostname": "10.0.0.67",
  "port": 5900,
  "connectionKey": "eb5e9d13-1caa-486b-bdda-130ca0d87df8"
}
...
}
```

Field	Data type	Details
connectionKey	String	Shared secret to authenticate the connection request from device side
hostname	Number	Endpoint on the local network to connect to
port	String	Port to be used on local network endpoint

CONNECTING TO A NEW ENDPOINT

For each `c8y_RemoteAccessConnect` operation the device agent receives, it opens a TCP client socket to the provided hostname and port. Using the provided ConnectionKey the agent also securely connects to the WebSocket endpoint on server side.

The following steps need to be implemented by the device agent when it receives a `c8y_RemoteAccessConnect` operation.

- The device sets the operation status to EXECUTING.
- The device establishes a connection to the Cumulocity Cloud Remote Access WebSocket at the following URL:

```
wss://<hostname>/service/remotearraccess/device/<connectionKey>
```

- The device sets up a TCP connection to the given hostname and port. Depending on the protocol (VNC, Telnet, SSH) the device will initiate a protocol-specific handshake. All data should be forwarded directly to the WebSocket endpoint (if already established).
- The operation status is set to SUCCESSFUL or FAILED based on the status of the previous steps.

OPERATING A CONNECTED ENDPOINT

When both connections are established and fully functional the agent simply must forward all binary packets between the TCP connection and the WebSocket in both directions.

DISCONNECTING AN ENDPOINT

Whenever one of the connections is terminated (WebSocket or TCP) the device agent should consider the session as ended and should also terminate both connections associated with the tunnel.

RECOMMENDATIONS

It is highly recommended to implement a small buffer especially for bootstrapping when one connection is already functional while the other is not setup yet.